

Лабораторна робота

Дослідження швидкодії операції `std::count_if`

Варіант 4 - `count_if`

Виконав: Дмитро Демчук, група К-25
2026

1. Завдання

Дослідити швидкодію операції `std::count_if` над послідовностями різного обсягу: (1) без політики виконання, (2) з різними політиками виконання (`seq`, `par`, `par_unseq`), (3) власним паралельним алгоритмом, що розбиває дані на K частин, обробляє кожну в окремому потоці послідовним `count_if`, а результат зводить `std::reduce`. Дослідити залежність часу від K , знайти найкраще K та порівняти його з кількістю апаратних потоків. Дослідження виконати в режимі `RELEASE` з двома рівнями оптимізації: без оптимізації (`-O0`) та з максимальною (`-O3`).

2. Методика

2.1. Компілятор і збірка

GCC (Homebrew `g++-16`), стандарт C++20. На macOS компілятор за замовчуванням - Apple Clang, але його стандартна бібліотека `libc++` не підтримує політики виконання `std::execution::par/seq/unseq/par_unseq` - код з ними не компілюється. Тому збірка виконана через GCC, чия бібліотека `libstdc++` реалізує ці політики через Intel TBB.

Встановлення залежностей і збірка (README.md):

```
brew install gcc tbb
make run
```

Команда `make run` збирає програму двічі - з `-O0` і з `-O3` (Makefile викликає `g++-16` з прапорцями `-std=c++20 -linclude -ltbb>/include -L<tbb>/lib -ltbb -pthread`) та записує вивід обох збірок у `results/output_O0.txt` і `results/output_O3.txt`.

2.2. Дані

Тестові послідовності - випадкові цілі числа в діапазоні $[2, 100000]$ (`std::mt19937` та `std::uniform_int_distribution`), що зберігаються в текстових файлах `data/seq_N.txt`. Досліджено три обсяги: $N = 100\,000$; $1\,000\,000$; $10\,000\,000$. Окремої програми для генерації немає - це робить клас `DataSet` усередині основної програми: якщо файли вже є, вони використовуються (тому режими `-O0` і `-O3` працюють з однаковим набором даних); якщо файлів немає - програма створює їх сама (`seed=42`); параметр запуску `--new` створює нові дані з випадковим `seed`.

2.3. Предикати

Використано два предикати різної «ваги»: `isEvenNumber` ($x \% 2 == 0$) - дешевий, одна арифметична операція; `isPrime` - дорогий, перевірка простоти пробними діленнями до $\sqrt{x}$.

2.4. Вимірювання часу

Для кожного дослідження фіксуються два таймери (`std::chrono::high_resolution_clock`): `total` - читання файлу з диска, завантаження в RAM, обчислення та вивід результату разом; `compute` - лише сам виклик `count_if` на вже завантажених у RAM даних. Різниця між ними показує, яку частку часу займає файловий ввід/вивід, а не сам алгоритм.

3. Частини 1-2: бібліотечний count_if без політики та з політиками

Час обчислення (compute, мс) для кожного режиму. Результат (кількість елементів, що задовольняють предикат) співпав у всіх режимах для однакового N і предиката - це підтверджує коректність усіх варіантів.

N	предикат	none	seq	par	par_unseq
100 000	even	0.127	0.125	0.047	0.031
100 000	prime	3.578	3.277	0.689	0.615
1 000 000	even	0.937	0.948	0.171	0.190
1 000 000	prime	26.816	26.693	5.925	5.959
10 000 000	even	9.385	9.576	1.604	1.764
10 000 000	prime	274.383	275.414	58.289	58.404

Таблиця 1. Час обчислення (-O3), мс.

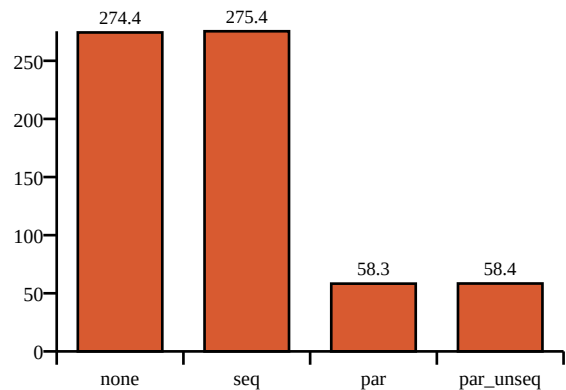
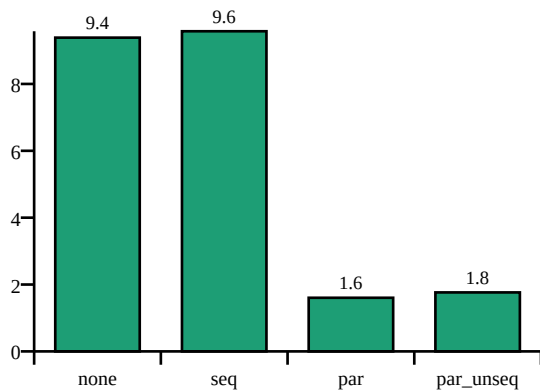


Рис. 1-2. Час обчислення на N=10 000 000 (-O3): зліва - дешевий предикат even, справа - дорогий предикат prime.

Висновок: на дорогому предикаті (prime) par/par_unseq прискорюють count_if приблизно у 4.7 раза (275.4 мс → 58.3 мс) на 8 апаратних потоках: роботи на елемент багато, і накладні витрати на розподіл між потоками відносно малі. На дешевому предикаті (even) виграш є (у 6.0 раза на N=10 000 000), але менший і нестабільніший - самого обчислення мало, тому помітну частку становлять накладні витрати. На малих N (100 000) час настільки малий (соті частки мс), що вимірювання залежить від шуму. Різниця між «без політики» і seq - в межах похибки вимірювання: обидва варіанти однопотокові.

Total проти compute

Таблиця 2 показує обидва таймери для N=10 000 000 (-О3). Для дешевого предиката майже весь total - це читання текстового файлу (розбір чисел через оператор >>), а не сам count_if; для дорогого частка обчислення помітно більша.

предикат	політика	total, мс	compute, мс	частка поза обчисленням
even	none	295.7	9.4	97%
even	seq	289.2	9.6	97%
even	par	283.9	1.6	99%
even	par_unseq	281.9	1.8	99%
prime	none	552.1	274.4	50%
prime	seq	550.8	275.4	50%
prime	par	339.6	58.3	83%
prime	par_unseq	335.9	58.4	83%

Таблиця 2. Total і compute, N=10 000 000, -О3.

4. Частина 3: власний паралельний алгоритм

Масив ($N=10\,000\,000$, предикат `prime`) ділиться на K приблизно однакових частин; кожна частина обробляється в окремому `std::thread` послідовним `count_if`, а часткові результати зводяться `std::reduce`. `std::thread::hardware_concurrency()` на тестовій машині = 8.

K	compute, мс (-O3)	прискорення (-O3)	compute, мс (-O0)	прискорення (-O0)
1	266.180	1.00	717.922	1.00
2	137.498	1.94	380.415	1.89
4	71.240	3.74	191.102	3.76
6	64.704	4.11	153.915	4.66
8	59.882	4.45	127.663	5.62
12	66.641	3.99	128.590	5.58
16	59.515	4.47	128.714	5.58
24	60.709	4.38	126.270	5.69
32	60.866	4.37	127.126	5.65

Таблиця 3. Залежність часу від K (прискорення = $T(1) / T(K)$).

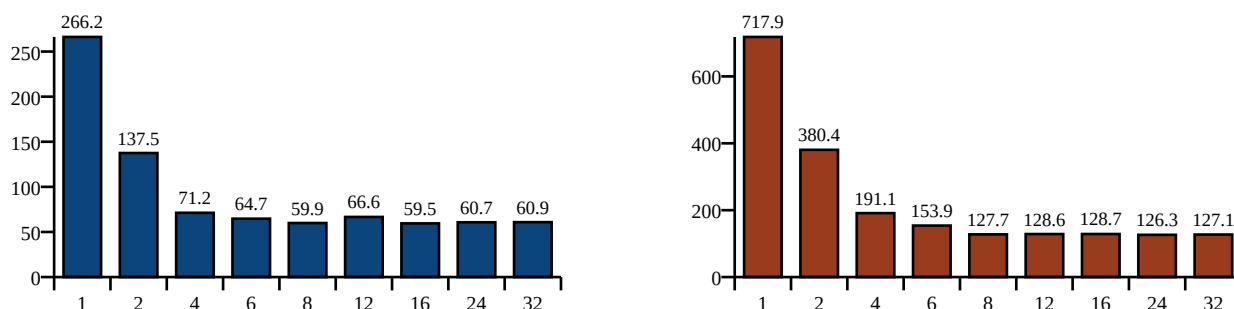


Рис. 3-4. Час власного паралельного алгоритму залежно від K (зліва -O3, справа -O0).

Найкраще K . Формально мінімум часу в таблиці: $K = 16$ (59.5 мс) на -O3 і $K = 24$ (126.3 мс) на -O0 (2.0 і $3.0 \times \text{hardware_concurrency} = 8$). Але вже при $K = 8$ (-O3) і $K = 8$ (-O0) час відрізняється від мінімуму лише на 0.6% і 1.1% - це в межах розкиду повторних запусків (у різних прогонах формальне найкраще K потрапляло на різні значення - 16, 24, 32). Тому практично найкраща швидкість досягається коли K дорівнює кількості апаратних потоків (8); подальше збільшення K виграшу не дає.

Закон зміни часу. Від $K=1$ до $K=4$ час падає майже обернено пропорційно K (подвоєння K - приблизно удвічі менший час: $266 \rightarrow 137 \rightarrow 71$ мс на -O3; $718 \rightarrow 380 \rightarrow 191$ мс на -O0), тобто $T(K) \approx T(1)/K$. Далі виграш швидко зменшується, і при K від 8 (-O3) і 8 (-O0) крива виходить на плато (60-67 мс на -O3, 126-129 мс на -O0). Максимальне прискорення становить $4.5 \times$ (-O3) і $5.7 \times$ (-O0), тобто менше за кількість апаратних потоків (8): паралельна частина перестає масштабуватися, коли всі апаратні потоки зайняті (ймовірно, ще й тому, що не всі вони рівноцінні за продуктивністю). Збільшення часу через накладні витрати на створення потоків (воно росте приблизно лінійно з K) при $K \leq 32$ непомітне на тлі часу роботи самих частин.

5. Вплив рівня оптимізації (-O0 проти -O3)

Порівняння тих самих операцій (N=10 000 000, predicate=prime, час compute) у збірках без оптимізації та з максимальною:

Режим	-O0, мс	-O3, мс	прискорення
без політики	711.3	274.4	2.59 ×
seq	710.9	275.4	2.58 ×
par	124.3	58.3	2.13 ×
власний алгоритм, K=8	127.7	59.9	2.13 ×

Таблиця 4. Вплив рівня оптимізації.

Сам лише рівень оптимізації компілятора дає прискорення у 2.1-2.6 раза - без жодної зміни коду і без жодного додаткового потоку. Це важливо враховувати: якби послідовну версію збирали з -O0, а паралельну - з -O3, виміряне «прискорення від паралелізму» було б завищене, бо частина виграшу походила б від оптимізації компілятора, а не від розподілу роботи між потоками. Тому політики виконання і власний алгоритм порівнювалися в межах одного рівня оптимізації (окремо для -O0 і окремо для -O3).

6. Висновки

- Політики par і par_unseq суттєво прискорюють count_if, коли предикат «важкий» (isPrime): у 4.7 раза на 8 апаратних потоках (N=10 000 000, -O3). На дешевому предикаті (isEvenNumber) виграш менший, бо накладні витрати розподілу роботи стають відносно вагомими.
- Різниця між «без політики» і seq несуттєва - обидва варіанти виконуються одним потоком.
- Власний алгоритм (розбиття на K потоків + std::reduce) поводить як бібліотечний par: при малих K час падає як $T(1)/K$, далі виходить на плато. Практично найкраща швидкість досягається коли K дорівнює кількості апаратних потоків (8); формальний мінімум (K = 16 на -O3) від цього не відрізняється в межах похибки і від запуску до запуску змінюється.
- Рівень оптимізації (2.1-2.6 раза) впливає на швидкість не менше, ніж паралелізм, тому порівнювати політики коректно лише в межах одного рівня оптимізації.

Повний вивід програми для обох рівнів оптимізації наведено у файлах results/output_O0.txt та results/output_O3.txt, що додаються до звіту разом з вихідним кодом (include/, src/, Makefile).

7. Додаток: вивід терміналу

7.1. Збирання виконуваного файлу (вивід терміналу)

```
king@Dmytros-MacBook-Pro lr_2 % make run
/opt/homebrew/bin/g++-16 -std=c++20 -O0 -Iinclude -I/opt/homebrew/opt/tbb/include src/CountAlgorithm.cpp
src/DataSet.cpp src/Experiment.cpp src/Predicates.cpp src/main.cpp -o lab_00 -L/opt/homebrew/opt/tbb/lib -ltbb
-pthread
/opt/homebrew/bin/g++-16 -std=c++20 -O3 -Iinclude -I/opt/homebrew/opt/tbb/include src/CountAlgorithm.cpp
src/DataSet.cpp src/Experiment.cpp src/Predicates.cpp src/main.cpp -o lab_03 -L/opt/homebrew/opt/tbb/lib -ltbb
-pthread
mkdir -p results
./lab_00 > results/output_00.txt
./lab_03 > results/output_03.txt
```

Рис. 5. Вивід терміналу: збирання і запуск командою make run.

7.2. Результат роботи програми (вивід терміналу)

```
king@Dmytros-MacBook-Pro lr_2 % ./lab_03
hardware_concurrency = 8
data: using existing files from data/

N=100000 predicate=even policy=none result=50016 total=3.651ms compute=0.127ms
N=100000 predicate=even policy=seq result=50016 total=3.744ms compute=0.125ms
N=100000 predicate=even policy=par result=50016 total=4.823ms compute=0.047ms
N=100000 predicate=even policy=par_unseq result=50016 total=3.638ms compute=0.031ms
N=100000 predicate=prime policy=none result=9435 total=6.808ms compute=3.578ms
N=100000 predicate=prime policy=seq result=9435 total=7.114ms compute=3.277ms
N=100000 predicate=prime policy=par result=9435 total=4.095ms compute=0.689ms
N=100000 predicate=prime policy=par_unseq result=9435 total=3.958ms compute=0.615ms

N=1000000 predicate=even policy=none result=500203 total=30.872ms compute=0.937ms
N=1000000 predicate=even policy=seq result=500203 total=28.94ms compute=0.948ms
N=1000000 predicate=even policy=par result=500203 total=28.388ms compute=0.171ms
N=1000000 predicate=even policy=par_unseq result=500203 total=28.113ms compute=0.19ms
N=1000000 predicate=prime policy=none result=96284 total=54.81ms compute=26.816ms
N=1000000 predicate=prime policy=seq result=96284 total=54.341ms compute=26.693ms
N=1000000 predicate=prime policy=par result=96284 total=34.22ms compute=5.925ms
N=1000000 predicate=prime policy=par_unseq result=96284 total=34.141ms compute=5.959ms
```

Рис. 6. Вивід ./lab_03: початок (N = 100 000 і N = 1 000 000).

```
N=10000000 predicate=even policy=none result=4998689 total=295.741ms compute=9.385ms
N=10000000 predicate=even policy=seq result=4998689 total=289.235ms compute=9.576ms
N=10000000 predicate=even policy=par result=4998689 total=283.904ms compute=1.604ms
N=10000000 predicate=even policy=par_unseq result=4998689 total=281.95ms compute=1.764ms
N=10000000 predicate=prime policy=none result=958677 total=552.118ms compute=274.383ms
N=10000000 predicate=prime policy=seq result=958677 total=550.779ms compute=275.414ms
N=10000000 predicate=prime policy=par result=958677 total=339.578ms compute=58.289ms
N=10000000 predicate=prime policy=par_unseq result=958677 total=335.918ms compute=58.404ms

custom parallel count_if, N=10000000 predicate=prime
  K    compute_ms    result
  1     266.180     958677
  2     137.498     958677
  4      71.240     958677
  6      64.704     958677
  8      59.882     958677
 12      66.641     958677
 16      59.515     958677
 24      60.709     958677
 32      60.866     958677
best K = 16 (compute=59.515 ms), hardware_concurrency = 8, K / hardware_concurrency = 2.00
```

Рис. 7. Вивід ./lab_03: продовження (N = 10 000 000 і частина 3).